

ToSh Cheat Sheet: Shell Basics

REPL workflow, help, capture forms, redirection, and jobs

spec + parser/tests

Start Here

Command	What it does
<code>dotnet run -project src/Tosh.Cli</code>	Start the ToSh REPL.
<code>help ls</code>	Explain one command or topic.
<code>help search regex</code>	Search the help catalog.
<code>help browse grep</code>	Open the full-screen help browser.
<code>time { ls -la }</code>	Measure wall time, CPU, and memory.
<code>history search ls</code>	Search command history by text.

```
# typed object pipeline
ls -la | where _.Type == file | sort Size | reverse | first 5
```

```
# shell-friendly discovery
help which
apropos json
```

Mental Model

- Pipelines carry real CLR objects, not plain text by default.
- The current pipeline item is `_` inside `where`, `each`, `map`, and similar blocks.
- `$tosh.Last.Result` stores the last successful statement result.

Rule of thumb: text is something you opt into with commands like `raw`, `to`, `join-lines`, `$(...)` or external programs.

Capture Forms

Form	Meaning
<code>(...)</code>	Evaluate a pipeline/expression and require exactly one object value.
<code>\$(...)</code>	Capture text output; multiple values join with newlines.
<code><(...)</code>	Materialize a pipeline to a temp file and pass its path-like handle.
<code>>(...)</code>	Send output to a process-substitution sink.

```
var user = (whoami)
var text = $(echo one two three)
var names = <(ls | get Name | sort)
/bin/cat $names
```

`(...)` is strict: if the inner pipeline yields 0 or more than 1 value, ToSh raises a diagnostic.

Session Commands

Command	Notes
<code>pwd</code> , <code>cd</code> , <code>dirs</code>	Current directory and stack navigation.
<code>back</code> , <code>forward</code>	Move through the directory stack.
<code>env</code> , <code>export</code> , <code>unset</code>	Read or change environment/session names.
<code>vars</code>	Show visible variables in scope.
<code>which ls</code>	Resolve a command name.
<code>clear</code>	Clear the terminal display.

```
pwd
cd src
dirs
back
env USER
```

Redirection

Syntax	Meaning
<code>out> file</code>	stdout to file, truncate
<code>out» file</code>	stdout to file, append
<code>err> file</code>	stderr to file, truncate
<code>o+e> file</code>	stdout and stderr together
<code><< text</code>	here-string input

```
echo "hello" out> hello.txt
echo "world" out»> hello.txt
/bin/sh -c "echo boom >&2" err> errors.txt
grep toast <<< "toast shell"
```

ToSh uses named redirection because `<` and `>` are also comparison operators in expressions.

Jobs And Async Work

```
/bin/sleep 5 &
jobs
wait-for 1
kill 1
```

- A trailing `&` backgrounds a pipeline or external command.
- `jobs` returns typed job records you can filter or inspect.
- `wait-for` waits on job ids or piped jobs.

At the *start* of an expression, `&name` means “function reference”, not “background job”.

Useful One-Liners

```
ls -la | where _.Type == file | get { Name, Size }
ip addr | where { _.State == up }
df | summarize _.Used
```

```
guid new v7
time { ls -la }
```

ToSh Cheat Sheet: Shell Basics

Interactive browsers, history, prompt modules, and live config

Help And Discovery

```
help --cli regex
help related ls
help categories
apropos pipeline
which ls grep tosh
types list
```

- `help -cli` opens the inline fuzzy browser.
- `help browse` opens the full-screen split-pane browser.
- `types` searches both CLR types and ToSh shell types.

Config And Display

```
config browse prompt
config get prompt.name-text
config set prompt.name-text toast
config reset display
config reload
```

```
view detail
view compact
view columns
view size bytes
view datetime table unix
```

`config` manages live shell settings and startup config; `view` is the quick way to tune rendering for the current session.

History Workflow

```
history
history path
history search grep
history expand 237
history run 237
history save
```

- REPL sugar: `!!`, `!237`, `!-2`, `!?text?`, `^old^new^`
- `history delete <spec>` removes entries; `reload` re-reads the file-backed store.

Directory Stack

```
dirs
dirs goto 2
dirs remove 1
dirs clear
```

- `dirs` is handy when you bounce among a few working roots.
- Use `goto` for fast return without retyping long paths.

Inspecting Objects

```
ls -la | first | inspect
new System.Random() | inspect -a
new System.Random() | inspect --flat
name-of (pwd)
type-of (date now)
```

```
ls -la | first | get-props
ls -la | first | get-prop Name
has-prop (pwd) FullName
has-method (pwd) ToString
```

In the REPL, `F1` opens help for the token under the cursor, `F2` tries inline inspect, and `Alt+H` / `Alt+I` are fallback bindings.

Prompt Modules

Module	What it adds
<code>prompt-dir</code>	current directory
<code>prompt-git</code>	branch and repo status
<code>prompt-time</code>	current wall clock time
<code>prompt-userhost</code>	user and host name
<code>prompt-history</code>	next history id
<code>prompt-jobs</code>	current background job count
<code>prompt-text</code>	literal styled text
<code>prompt-newline</code>	multi-line prompt split

```
prompt-userhost --dim
prompt-dir --depth 2 --fg blue
prompt-git --fg bright-green
prompt-time --format "HH:mm"
```

Run Files And Processes

```
source ./script.tosh
source ./script.tosh ...$tosh.Script.Args
env DEBUG=1 -- dotnet test
exec zsh
exit
```

Jobs And Waiting

```
sleep 5s &
jobs
wait-for $tosh.Last.Result.Id
kill 3
```

- `jobs` and `wait-for` are the main background-job tools.
- `bg` and `fg` resume suspended jobs stopped with `Ctrl+Z`.

Runtime State

```
vars
env USER HOME
$tosh.Last.Result
$tosh.Script.Path
$tosh.Session.OpenHandleCount
```

- `$tosh.Last.Result` tracks the last successful statement result.
- `$tosh.Script.Path` is useful inside sourced or executed scripts.
- Open managed handles are visible through the `$tosh.Session` namespace.

Small Conveniences

```
assert ($env.HOME != null)
styled "warning" --fg yellow --bold
read-line "name> "
ignore
```

Tool	Why it matters
<code>assert</code>	fail early in scripts with a clear message
<code>styled</code>	emit colored/styled terminal text segments
<code>read-line</code>	prompt for one line of interactive text
<code>ignore</code>	explicitly discard pipeline output

Config Object

```
$tosh.Config.Prompt.NameText = "toast"
$tosh.Config.Repl.ContinuationPrompt = "...> "
$tosh.Config.Display.StorageSize.Mode = "Bytes"
config get prompt
```

- Direct `$tosh.Config` edits affect the live session immediately.
- Use `config reload` to replay startup config files into the session.

`source` reruns a file every time in the current scope; `require` caches modules once per session; `exec` replaces the current ToSh process with another program.